Kourai Khryseai: Transparent Human-on-the-Loop Multi-Agent Software Development

An interpretability lens on agent-based software engineering

AJ Barea • Rochester Institute of Technology • Advisor: Dr. Leon Reznik

NE Agents Day 2026 • Jane Street, New York • May 8, 2026

KEYWORDS: AI agents • multi-agent systems • software engineering • human-on-the-loop • observability

ABSTRACT & RESEARCH QUESTIONS

Multi-agent SE assistants are increasingly deployed, yet the coordination layer between specialists is hidden — developers can't see, study, or steer how these systems respond to prompts. *Kourai Khryseai* exposes that layer: an orchestrator routes requests across specialist agents while streaming state, pausing on ambiguity, and bounding the repair loop against ping-pong oscillation. The result is an **observable, interpretable, and steerable instrument** for AI4SE researchers and dev-tool builders.

- RQ1 · MONITOR — What mechanisms expose the coordination of multi-agent SE assistants?
- RQ2 · COMMUNICATE — What mechanisms make multi-agent decisions legible to a developer-supervisor?
- RQ3 · CONTROL — What mechanisms let a developer-supervisor steer agent behavior at the right grain?

INTRODUCTION & RELATED WORK

SWE-bench [3] and OpenHands [5] measure issue-resolution. Magentic-One [2] orchestrates specialists; HULA [4] keeps humans in every step; AgentTrace [1] traces single agents. **Kourai combines all four**: orchestration, streamed state, HOTL supervision.

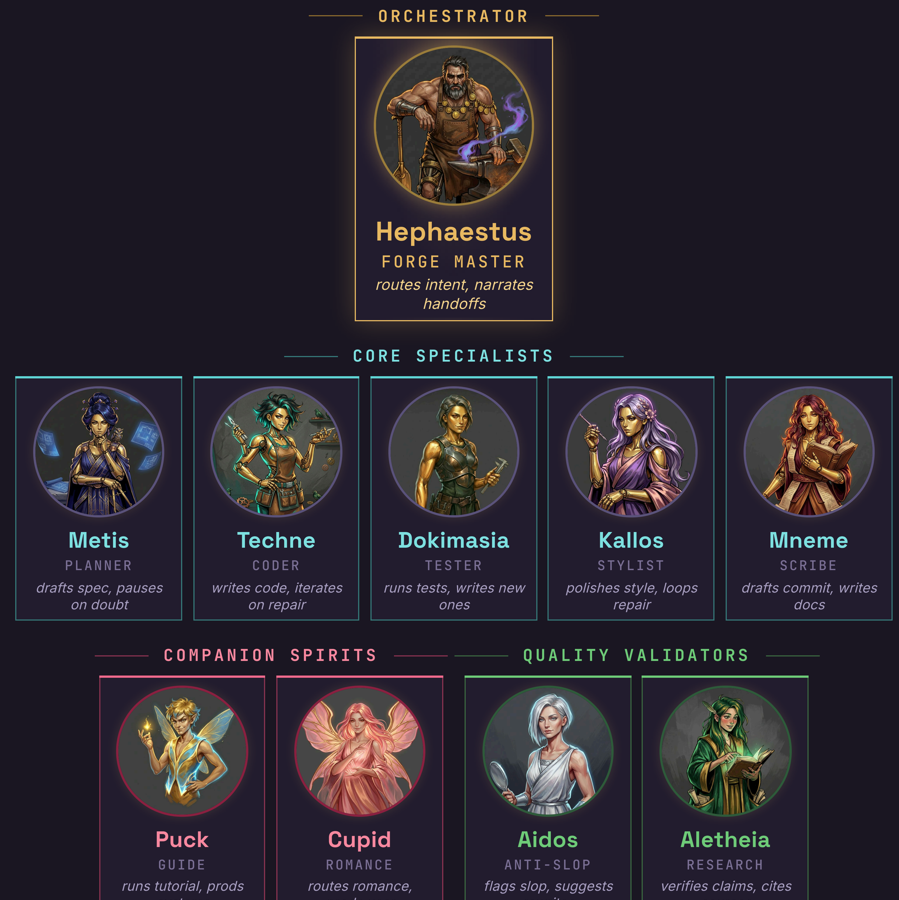
SYSTEM	MULTI-AGENT	STREAMED	HOTL
SWE-bench [3]	–	–	–
OpenHands [5]	✓	partial	–
Magentic-One [2]	✓	–	–
HULA [4]	–	✓	HITL
AgentTrace [1]	–	✓	–
Kourai	✓	✓	HOTL

HITL · HOTL · HOOTL — A SPECTRUM

HITL	HOTL (KOURAI'S DESIGN POINT)	HOOTL
HUMAN-IN-THE-LOOP	HUMAN-ON-THE-LOOP	HUMAN-OUT-OF-THE-LOOP
human *authors* each step; agent suggests.	agent acts autonomously; human *supervises & steers*.	agent runs *without* human supervision.
HULA [4], Copilot suggest mode	root: Endsley 1995; AI/SE: Cleland-Huang et al. 2020	AutoGPT, fully agentic pipelines

Multi-agent SE belongs at HOTL: per-step decisions too frequent for HITL, too consequential for HOOTL. Kourai operationalizes it via streamed state, `AgentInputRequired` pauses, and bounded repair against ping-pong oscillation.

THE FORGE PARTY — 10 AGENTS



INTERPRETABILITY MECHANISMS — MONITOR · COMMUNICATE · CONTROL

Three pillars of the instrument; each enumerated mechanism appears elsewhere on the poster as a first-class system feature.

MONITOR (RQ1) — *see what's happening*
- streamed status messages
- OTel `gen_ai.*` spans
- Forge Transcript SQLite
- Jaeger traces & metrics

COMMUNICATE (RQ2) — *system explains itself*
- in-character Hephaestus handoffs
- named, color-coded specialists
- routing-by-intent templates
- per-agent role labels

CONTROL (RQ3) — *steer at the right grain*
- `AgentInputRequired` pause
- Kallos⇅Techne bounded repair
- `MAX_ITERATIONS` against ping-pong
- graceful skip on unreachable

SYSTEM DESIGN — ORCHESTRATED SPECIALIST PIPELINE

CLI REPL (make cli) · PYGAME GUI (make gui) · REN'PY VN (make vn)

A2A • HTTP • SSE STREAMING

HEPHAESTUS — FORGE_MASTER • ROUTE( ) • FORGE_TRANSCRIPT

01 Metis (PLANNER) → 02 Techne (CODER) → 03 Dokimasia (TESTER) → 04 Kallos (STYLIST) → 05 Mneme (SCRIBE)

<repair/> Kallos → Techne • MAX_ITERATIONS = 3

streamed status throughout · ambiguity → `AgentInputRequired` pause · unreachable → graceful skip

ROUTING BY INTENT — 7 PIPELINE TEMPLATES

implement x	metis → techne → dokimasia → kallos → mneme
fix bug x	techne → dokimasia → kallos → mneme
add tests for x	dokimasia → kallos → mneme
clean up x	kallos → mneme
commit prep	mneme
plan x	metis
lint/format x	kallos

FORGE MASTER NARRATES — IN-CHARACTER HANDOFFS

Each handoff emits an in-character line from Hephaestus, making orchestration legible as dialogue.

→ METIS	*"Draw up the plans. And no improvising."*
→ TECHNE	*"Write something worthy of my forge."*
→ DOKIMASIA	*"Find every flaw. Leave nothing."*
→ KALLOS	*"Go make it pretty or whatever you do."*
→ MNEME	*"Write it down. The FACTS, not your opinions."*

STREAMED HOTL — CLI SAMPLE

```
> implement CSV export with tests
🔥 Hephaestus: Metis! Draw up the plans.
📐 Metis: analyzing...
   → streaming for >100MB files? (y/n)
> yes, chunked I/O
📐 Metis: spec ready ✓
⚙ Techne: writing files... ✓
🧪 Dokimasia: 12/12 passing ✓
✨ Kallos: no issues ✓
📜 Mneme: commit drafted ✓
```

ONE BACKEND • THREE HOSTS

Same orchestration logic under all three. Interface varies — terminal REPL, pygame chat with TTS voices, illustrated visual novel — while backend logic stays fixed. A platform for studies of UI-agnostic agent behavior.

CLI REPL — status stream · keyboard checkpoint

```
Kourai Khryseai — Golden Maidens
Forged in fire, refined by hand

Enter send  ·  Ctrl+V paste  ·  Alt+V attach image
/help commands  ·  /q quit

Connected to Hephaestus v0.9.2 · 10 agents online · forge warm
Model: claude-opus-4-7 · demo mode (no network, no LLM)

> implement CSV export with tests for the events module

🔥 Hephaestus: "Metis! Draw up the plans. And no improvising."

📐 Metis: analyzing events module...
   └ found 3 models with datetime fields
   └ nested relationships in Attendance and Session

📐 Metis: "One decision before I draft the spec."

  → "Should CSV export stream chunked I/O for large files?"
    events.json has ~420k rows in prod. Loading all at once
    would spike memory. Streaming stays constant-memory.

    [y] yes, stream chunked      [n] no, load in memory
    [?] explain trade-offs       [^C] cancel

> ▌
```

PYGAME GUI — portrait chat · free-form input

REN'PY VN — HANDOFF — routing spoken aloud · agents named inline

REN'PY VN — CHECKPOINT — ambiguity becomes a menu choice

ARTIFACT EVALUATION — TEST PROPERTIES

PROPERTY	TEST FOCUS	✓
COM Pipeline ordering	end-to-end canonical route	✓
MON Forge Transcript accumulation	shared context per specialist	✓
CTL Clarification loops	`AgentInputRequired` pause/resume	✓
CTL Bounded Kallos⇅Techne repair	`MAX_ITERATIONS = 3`	✓
CTL Graceful degradation	"Skipped (unreachable)"	✓

Every claim in the MCC panel has a test backing it.

CONCLUSION

Transparency is a systems property, not just a UX choice.

Across **Monitor**, **Communicate**, and **Control**, Kourai shows that multi-agent SE coordination can be made *observable*, *legible*, and *steerable* — a platform for future studies of agent memory, orchestration, and supervisor trust.

REFERENCES

[1] A. AlSayyad, K. Y. Huang, R. Pal. *AgentTrace: A structured logging framework for agent system observability.* arXiv:2602.10133, 2026.

[2] A. Fourney et al. *Magentic-One: A generalist multi-agent system for solving complex tasks.* arXiv:2411.04468, 2024.

[3] C. E. Jimenez et al. *SWE-bench: Can language models resolve real-world GitHub issues?* ICLR 2024.

[4] W. Takerngsaksiri et al. *Human-in-the-loop software development agents.* ICSE-SEIP 2025.

[5] X. Wang, B. Li, Y. Song et al. *OpenHands: An open platform for AI software developers as generalist agents.* ICLR 2025.

OBSERVABILITY — END-TO-END TRACING

one user request → spans across the specialist pipeline — HOTL pauses and repair loops are deliberate, not edge cases

A2A 10 HTTP services · **MCP** shell • memory • context7 · **SQLITE** Forge Transcript · **COMPOSE** Starlette • 14 services

OTEL SDK → OTLP COLLECTOR → JAEGER traces · PROMETHEUS metrics